ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

NHẬP MÔN TRÍ TUỆ NHÂN TẠO (CO3061)

Bài tập lớn 1: Tìm kiếm

Giảng viên: Vương Bá Thịnh
Nhóm: L03

STT	Họ và tên	MSSV
1	Vũ Minh Sang	2312944
2	Nguyễn Đình Khang	2411451
3	Nguyễn Đình Khôi	2311681
4	Đậu Minh Khôi	2311659

Thành phố Hồ Chí Minh, tháng 3 năm 2026

Mục lục

Danh sách thành viên và nhiệm vụ **2**

1 Trò chơi Lều (Tents) **3**

1.1 Giới thiệu 3

1.2 Thuật toán tìm kiếm theo chiều sâu Depth-First Search (DFS) . . . 4

1.2.1 Ý tưởng 4

1.2.2 Mô hình hóa bài toán 4

1.2.3 Hiện thực 5

1.3 Thuật toán tìm kiếm bằng kinh nghiệm 7

1.3.1 Ý tưởng 7

1.3.2 Mô hình hóa bài toán 7

1.3.3 Hiện thực 8

1.4 So sánh hiệu năng giữa 2 thuật toán: 10

2 Trò chơi Killer Sudoku **12**

2.1 Giới thiệu 12

2.2 Thuật toán tìm kiếm quay lui đơn giản (Naive Back-tracking) (Với Killer Sdoku kích thước 4×4) 13

2.2.1 Ý tưởng 13

2.2.2 Mô hình hóa 13

2.2.3 Hiện thực 14

2.3 Thuật toán tìm kiếm bằng kinh nghiệm 16

2.3.1 Ý tưởng 16

2.3.2 Mô hình hóa bài toán 16

2.3.3 Cải tiến so với thuật toán Naive 16

2.3.4 Hiện thực 16

2.4 So sánh hiệu năng giữa 2 giải thuật 18

Danh sách thành viên và nhiệm vụ

STT	Họ và tên	MSSV	Nhiệm vụ	Hoàn thành
1	Vũ Minh Sang	2312944	Tìm kiếm có kinh nghiệm cho Killer Sudoku	100%
2	Nguyễn Đình Khang	2411451	Tìm kiếm mù cho Killer Sudoku	100%
3	Nguyễn Đình Khôi	2311681	Tìm kiếm mù cho Tents	100%
4	Đậu Minh Khôi	2311659	Tìm kiếm có kinh nghiệm cho Tents	100%

Bảng 1: Danh sách thành viên và nhiệm vụ

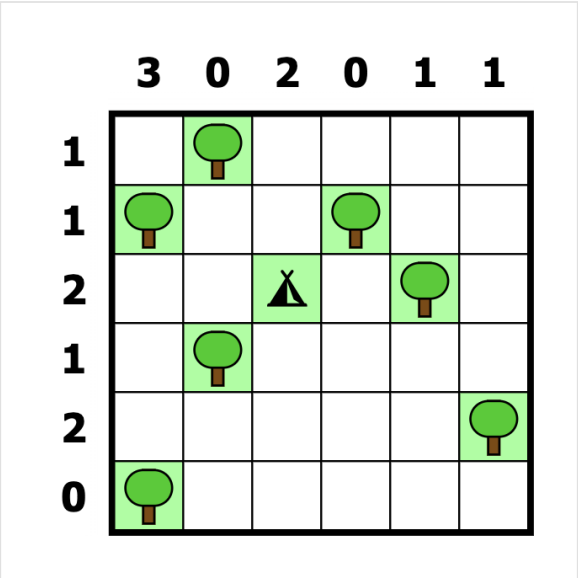
1 Trò chơi Lều (Tents)

1.1 Giới thiệu

Mô tả: Trò chơi **Lều (Tents)** là một trò chơi giải đố được cung cấp ở trang `https://www.puzzle-tents.com/`. Trò chơi không đòi hỏi kiến thức toán học phức tạp mà tập trung vào khả năng suy luận và thực thi các ràng buộc chặt chẽ để tìm ra lời giải duy nhất.

Thành phần trò chơi: Trò chơi là một ma trận vuông có lưới, gồm:

- **Cây (Trees):** Các cây được đặt cố định ở các ô của ma trận.
- **Lều (Tents):** Các lều được người chơi đặt vào các ô trống của ma trận sao cho thỏa mãn luật chơi.
- **Gợi ý số lượng (Clues):** Các con số nằm ở đầu mỗi hàng và mỗi cột, cho biết tổng số lều chính xác phải có trong hàng hoặc cột đó.



Hình 1: Hình ảnh trò chơi Tents

Luật chơi:

- **Quan hệ 1-1:** Mỗi cái cây phải được ghép cặp với đúng một chiếc lều duy nhất.
- **Vị trí của lều:** Một chiếc lều phải nằm ở ô kề cạnh (`trái`, `phải`, `trên`, `dưới`) của cái cây tương ứng với nó.

- **Khoảng cách giữa các lều:** Các chiếc lều không bao giờ được nằm cạnh nhau, kể cả theo đường chéo.
- **Khớp với gợi ý:** Tổng số lều ở mỗi hàng và mỗi cột phải bằng con số gợi ý đã cho ở đầu hàng hoặc cột đó.

1.2 Thuật toán tìm kiếm theo chiều sâu Depth-First Search (DFS)

1.2.1 Ý tưởng

Thuật toán thực hiện duyệt qua từng cây một. Với mỗi cây, thử đặt lều ở một trong bốn hướng khả thi `(trái, phải, trên, dưới)`. Nếu việc đặt lều không vi phạm các ràng buộc của trò chơi, thuật toán sẽ chuyển sang cây tiếp theo bằng đệ quy. Nếu đến một bước nào đó mà không có hướng nào để đặt lều cho cây đang xét, thuật toán sẽ quay lại cây trước đó, loại bỏ lều đã đặt và thử hướng khác.

1.2.2 Mô hình hóa bài toán

Trạng thái tổng quát (Global State):

- **Thành phần cố định:** Tọa độ của các cây `[(hàng,cột),...]` và gợi ý số lượng lều ở mỗi hàng và mỗi cột `[x1,x2,...]`.
- **Thành phần thay đổi:** Danh sách tọa độ các lều `[(hàng,cột),...]` đã được đặt vào ma trận ở một thời điểm bất kỳ.
- **Chỉ số cây:** Chỉ thứ tự của cây hiện tại đang xét.

Trạng thái bắt đầu (Initial State):

- Ma trận chỉ chứa các các cây và các số gợi ý ở mỗi hàng và mỗi cột.
- Không có lều nào ở trong ma trận (danh sách lều rỗng).
- Xét từ cây có chỉ số bằng 0.

Trạng thái kết thúc (Final State):

- Mỗi cây trong ma trận đã được ghép cặp với đúng một chiếc lều duy nhất.
- Tổng số lều ở mỗi hàng và mỗi cột phải bằng đúng với các con số gợi ý đã cho trước.

Các bước chuyển trạng thái (Transitions):
Bước chuyển là việc chuyển từ việc xét cây thứ i sang cây thứ $i+1$ bằng cách thực hiện các hành động sau:

1. **Chọn hướng:** Đặt một cái lều vào một trong bốn hướng (`trái, phải, trên, dưới`) xung quanh cái cây hiện tại đang xét.
2. **Kiểm tra ràng buộc:** Vị trí mới của lều phải thỏa mãn các ràng buộc sau:
   - Nằm trong ma trận và không nằm đè lên cây.
   - Không vượt quá số lượng lều được gợi ý của hàng và cột đó.
   - Không tiếp xúc với bất kỳ lều nào đã đặt trước đó trong phạm vi 8 ô xung quanh (`bắc, đông bắc, đông, đông nam, nam, tây nam, tây, và tây bắc`).
3. **Cập nhật:** Nếu vị trí mới thỏa mãn các ràng buộc trên, thực hiện bước chuyển (tức đặt lều vào vị trí đó và chuyển sang cây kế tiếp).
4. **Quay lui:** Nếu không tìm được vị trí hợp lệ để đặt lều cho cây kế tiếp, thuật toán sẽ quay lại cây trước đó, loại bỏ lều đã đặt và thử hướng khác.

1.2.3 Hiện thực

Môi trường hệ thống:

- **Ngôn ngữ:** Python 3 (`Python 3.14.3`).
- **Các thư viện sử dụng:** `time`, `tracemalloc`.

Mã nguồn:
Mã nguồn được thiết kế với lớp `TentsDFS`. Các phương thức chính của mã nguồn bao gồm:

- Phương thức khởi tạo: `__init__(self, rows, cols, trees, row_clues, col_clues)`.
- Phương thức `is_valid_position(self, row, col)`: Phương thức này kiểm tra xem ô `(row,col)` có vi phạm các quy tắc: không đè lên cây, không vượt quá số lều tối đa của hàng hoặc cột `(row_clues,col_clues)`, và không chạm vào các lều khác trong phạm vi 8 ô xung quanh.
- Phương thức `check_clues(self)`: Phương thức này duyệt lại toàn bộ ma trận để đảm bảo tổng số lều ở mỗi hàng và mỗi cột khớp hoàn toàn với các con số gợi ý ban đầu.

- Phương thức `solve(self, tree_index = 0)`: Phương thức này sử dụng đệ quy để duyệt qua danh sách các cây bắt đầu từ cây có thứ tự là 0. Với mỗi cây, phương thức sẽ thử đặt lều ở 4 hướng. Nếu phương thức `is_valid_position` trả về `True`, lều được thêm vào `self.tents`. Nếu không tìm được hướng nào phù hợp cho các bước tiếp theo, hàm thực hiện việc quay lui để thử lại.

Cách chạy chương trình:

- Input đầu vào:

```
    rows, cols = 8, 8
    row_clues = [2, 1, 1, 1, 1, 2, 2, 2]
    col_clues = [3, 0, 3, 1, 1, 1, 2, 1]
    trees = [(0, 7),
             (1, 1), (1, 3),
             (2, 1),
             (4, 4), (4, 6),
             (5, 0), (5, 3),
             (6, 1), (6, 6),
             (7, 3), (7, 6)]
```

- Lệnh chạy chương trình:
  - `python3 ten_file.py` (Unix).
  - `python ten_file.py` (Windows).
- Kết quả:

Hình 2: Kết quả của chương trình với input là ma trận (8×8)

1.3 Thuật toán tìm kiếm bằng kinh nghiệm

1.3.1 Ý tưởng

Bài toán thực hiện lần lượt đặt lều vào các ô trống. Trước khi đặt, thuật toán tìm và sắp xếp các ô có thể đặt lều vào một danh sách, theo thứ tự từ trên xuống và từ trái qua. Đây bao gồm các ô nằm cạnh các cây và các ô có các row hint và column hint khác 0. Bài toán thực hiện lần lượt đặt lều theo danh sách trên, loại bỏ các ô xung quanh lều. Khi lều được đặt, thuật toán thực hiện gọi đệ quy để kiểm tra vị trí tiếp theo. Khi xuống một hàng mới, thuật toán kiểm tra ràng buộc các hàng bên trên, nếu số lều trên hàng bị thiếu, thuật toán quay về về các hàng trước đó.

1.3.2 Mô hình hóa bài toán

Trạng thái tổng quát (Global State):

- **Thành phần cố định:**
  - Tọa độ của các cây `[(hàng,cột),...]`.
  - Gợi ý số lượng lều ở mỗi hàng và mỗi cột `[x1,x2,...]`.
  - Danh sách các vị trí có thể đặt.
- **Thành phần thay đổi:**
  - Ma trận 2D chứa các cây và các lều. Mỗi ô có một trong các giá trị `EMPTY` (trống, có thể đặt), `CANTPLACE` (không thể đặt), `TENT` (lều), `TREE` (cây).
  - Mảng chứa số lượng lều đang có trong mỗi cột.
  - Mảng chứa số lượng lều đang có trong mỗi hàng.
  - Vị trí đặt lều.
  - Lịch sử đặt lều.
- **Chỉ số lều**: Số thứ tự của lều sắp được đặt.
- **Chỉ số vị trí**: Vị trí đang được kiểm ra trong danh sách vị trí có thể đặt.
- **Chỉ số hàng hiện tại**: Chỉ số hàng của vị trí đặt thành công gần đây nhất.

Trạng thái bắt đầu (Initial State):

- Ma trận bao gồm cây và các số gợi ý ở mỗi hàng và mỗi cột.
- Không có lều nào ở trong ma trận.
- Danh sách vị trí chứa các ô có thể đặt.

- Lịch sử không có gì.
- Xét từ chỉ số lều bằng 0, chỉ số vị trí bằng 0, và chỉ số hàng hiện tại bằng 0.

Trạng thái kết thúc (Final State):

- Mỗi cây trong ma trận được ghép với đúng một lều duy nhất.
- Tổng số lều ở mỗi hàng và mỗi cột phải bằng với các con số gợi ý đã cho trước.

Các bước chuyển trạng thái (Final State):

1. **Chọn vị trí trong danh sách:** bắt đầu từ chỉ số vị trí ngay sau chỉ số vị trí của lều mới được đặt trước đó.
2. **Kiểm tra ràng buộc:**

   - Vị trí không nằm cạnh một cái lều đã được đặt (cạnh kề hoặc cạnh góc).

   - Vị trí phải trống (không phải lều).
3. **Commit:** Khi các ràng buộc trên được thỏa mãn, thuật toán thực hiện "Commit", lưu lại lịch sử vị trí đặt lều, và quay về bước 1.
4. **Quay lui:** Nếu như chỉ số hàng trong vị trí được chọn nằm ở một hàng mới (hàng chưa đặt lều nào), số lượng các lều trong mỗi hàng trước đó phải bằng với gợi ý các hàng tương ứng.

   Khi điều kiện không thỏa mãn, thuật toán ngay lập tức thoát khỏi lần gọi đệ quy hiện tại.
5. **Revert:** Thuật toán gỡ bỏ lều đã đặt và chọn vị trí kế tiếp trong danh sách.

1.3.3 Hiện thực

Môi trường hệ thống:

- **Ngôn ngữ**: Python 3.14.3.
- **Thư viện**: time, tracemalloc.

Mã nguồn:

- Phương thức khởi tạo:

```
__init__(self,
    rows,           # số hàng
    columns,        # số cột
    row_tents,      # gợi ý hàng
    column_tents,   # gợi ý cột
    trees,          # danh sách cây
    demo=True,
)
```

Thực hiện xây dựng ma trận bài toán và tạo danh sách các ô có thể đặt lều.

- Phương thức thêm lều:

```
try_add_tent(
    self,
    r: int,
    c: int
) -> Optional[tuple[int, int]]
```

Trả về vị trí thêm nếu vị trí là một ô trống, nằm cạnh một cây, không chạm vào các lều khác trong phạm vi 8 ô xung quanh, và không vượt quá số lượng lều trong cột và hàng:

- Phương thức

```
solve(self,
    tents_no=0, # chỉ số lều sắp được đặt.
    start=0,    # chỉ số vị trí bắt đầu trong danh sách.
    current_r=0 # Chỉ số hàng của lần đặt lều trước.
) -> bool
```

Hàm sử dụng đệ quy để duyệt qua danh sách các ô có thể đặt lều. Nếu như ta đi vào hàng mới, kiểm tra xem số lượng các lều ở hàng trên có đúng với gợi ý hay không. Nếu không, thoát khỏi phương thức. Sau khi đặt thành công, gọi lần đệ quy mới với tents_no thêm 1, start bằng chỉ số vị trí kế tiếp sau vị trí của lều mới được đặt, và current_r là chỉ số hàng của lều mới được đặt. Nếu lần gọi đệ quy con thành công, trả về thành công. Nếu không, gỡ lều mới được đặt và chọn vị trí tiếp theo trong danh sách.

Chỉ số lều bằng với số lượng lều tối đa tương đương với một lời giải.

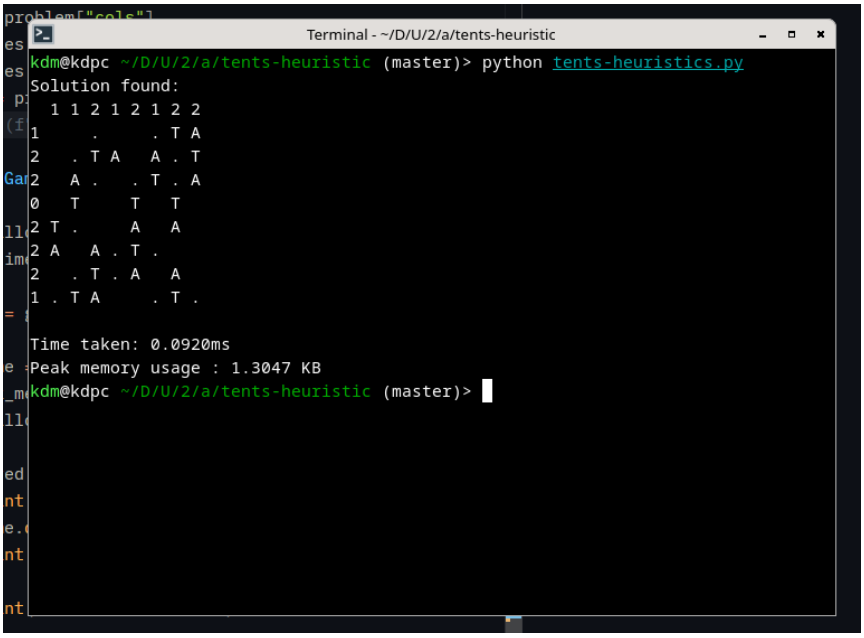
Chạy chương trình:

- Input đầu vào:

```
    rows, cols = 8, 8
    row_clues = [2, 1, 1, 1, 1, 2, 2, 2]
    col_clues = [3, 0, 3, 1, 1, 1, 2, 1]
    trees = [(0, 7),
             (1, 1), (1, 3),
             (2, 1),
             (4, 4), (4, 6),
             (5, 0), (5, 3),
             (6, 1), (6, 6),
             (7, 3), (7, 6)]
```

- Lệnh chạy chương trình: `python3 ten_file.py (Unix)` hoặc `python ten_file.py (Windows)`.

- Kết quả:



Kết quả của chương trình chạy

1.4 So sánh hiệu năng giữa 2 thuật toán:

Với đầu vào là 8x8, ta có kết quả so sánh chạy trung bình như sau:

	Time	Peak memory
DFS	$\sim 0.1151ms$	$\sim 1.3724KB$
Heuristic	$\sim 0.06ms$	$\sim 0.6771KB$

Với đầu vào 10x10:

	Time	Peak memory
DFS	~ 0.6209	$\sim 1.6511KB$
Heuristic	$\sim 0.9611ms$	$\sim 0.8645KB$

Nhận xét:

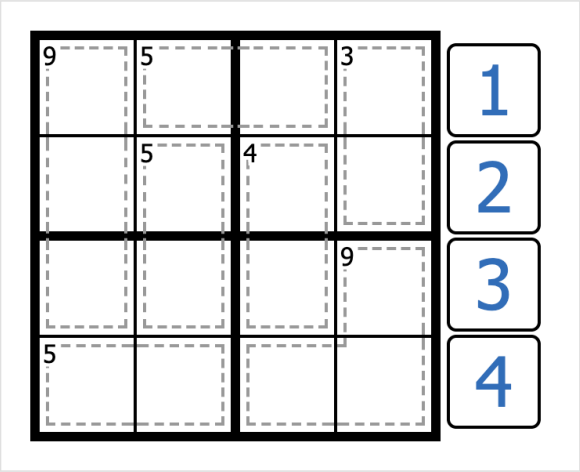
Thuật toán DFS với input 8x8 có xu hướng chậm hơn so với thuật toán Heuristic, nhưng nhanh hơn trong một vài Input 10x10. Thuật toán Heuristic cho thấy sử dụng ít bộ nhớ hơn so với thuật toán DFS trong cả hai trường hợp.

2 Trò chơi Killer Sudoku

2.1 Giới thiệu

Mô tả: Trò chơi **Killer Sudoku** là một biến thể của Sudoku truyền thống, được cung cấp tại trang `https://www.puzzle-killer-sudoku.com/`. Trò chơi này đòi hỏi sự kết hợp giữa tư duy logic của Sudoku và khả năng tính toán số học để thỏa mãn các ràng buộc về tổng số trong từng vùng xác định.
Thành phần trò chơi: Trò chơi thường là một lưới vuông kích thước N (Ví dụ như 4×4), gồm:

- **Ô (Cells):** Các ô trống cần điền các số từ 1 đến N.
- **Vùng (Cages):** Các nhóm ô được bao quanh bởi đường đứt đoạn. Mỗi vùng có một con số nhỏ ở góc trên bên trái.
- **Tổng của vùng (Cage Sum):** Con số nhỏ trong mỗi vùng chính là tổng của các số được điền vào các ô thuộc vùng đó.



Hình 3: Hình ảnh trò chơi Killer Sudoku

Luật chơi: (Với Killer Sdoku kích thước 4×4)

- **Luật Sudoku cơ bản:** Mỗi hàng, mỗi cột và mỗi khối 2×2 phải chứa các số từ 1 đến 4 mà không được lặp lại.
- **Ràng buộc tổng (Cage Constraint):** Tổng của tất cả các số trong một vùng (cage) phải bằng chính xác con số tổng được ghi ở góc vùng đó.
- **Lời giải duy nhất:** Tương tự các bài toán Logic Puzzles khác, mỗi bảng Killer Sudoku chỉ có một lời giải đúng duy nhất.

2.2 Thuật toán tìm kiếm quay lui đơn giản (Naive Backtracking) (Với Killer Sdoku kích thước 4×4)

2.2.1 Ý tưởng

Thuật toán chọn từng ô trong ma trận. Với mỗi ô, xét thử các ***tất cả*** giá trị khả thi. Nếu giá trị thử không vi phạm các điều kiện ràng buộc, thuật toán đệ quy sang ô tiếp theo. Nếu không có giá trị hợp lệ, quay lui.

2.2.2 Mô hình hóa

Biến và miền giá trị. Mỗi ô trong bảng Sudoku được biểu diễn bởi một tọa độ $X_{r,c}$ với $r, c \in \{0, 1, \ldots, 3\}$. Mỗi biến nhận giá trị trong tập $\{0, 1, 2, \ldots, 4\}$ (Số 0 đại diện cho việc ô đó chưa điền giá trị).

Tất cả các ô đều có cùng miền giá trị:

$$D_{r,c} = \{1, 2, \ldots, 4\}$$

Các ràng buộc: Bài toán bao gồm hai nhóm ràng buộc chính:

- **Ràng buộc Sudoku:**
  - Trên mỗi hàng: $X_{r,c_1} \neq X_{r,c_2}$ với $c_1 \neq c_2$
  - Trên mỗi cột: $X_{r_1,c} \neq X_{r_2,c}$ với $r_1 \neq r_2$
  - Trong mỗi khối 3×3: các giá trị không được trùng lặp
- **Ràng buộc cage:** Mỗi cage $C_k = (S_k, \{(r, c), \ldots\})$ bao gồm một tổng S_k và một tập các ô. Các biến trong cùng một cage phải thỏa mãn:

$$\sum_{(r,c) \in C_k} X_{r,c} = S_k$$

  và không có hai ô nào nhận cùng giá trị:

$$X_{r_i,c_i} \neq X_{r_j,c_j}, \quad \forall i \neq j$$

Trạng thái tổng quát của bài toán bao gồm

- Ma trận 4×4 chứa các giá trị từ 1 đến 4

- Tập miền giá trị hiện tại của từng biến

Trạng thái bắt đầu:

Tất cả các biến chưa được gán giá trị và có miền đầy đủ $\{1, \ldots, n\}$.

Trạng thái kết thúc:

Mọi biến đều được gán giá trị và tất cả các ràng buộc được thỏa mãn.

Quá trình tìm kiếm tương ứng với việc gán dần giá trị cho các biến, đồng thời loại bỏ các giá trị không còn hợp lệ, cho đến khi mỗi biến chỉ còn đúng một giá trị duy nhất.

Không gian tìm kiếm: Tổng quát Với một ma trận kích thước $N \times N$, kích thước không gian tìm kiếm là: $N^{(N^2)}$.

2.2.3 Hiện thực

Môi trường hệ thống:

- **Ngôn ngữ**: `Python 3.12.7`.
- **Thư viện**: `math, sys, time, tracemalloc, itertools`

Source code:

Được thiết kế với class `Blind_Grid`. Một `Blind_Grid` bao gồm nhiều class dữ liệu nhỏ hơn, như:

- `Blind_Cage` để lưu thông tin của một vùng (Cage)
- Trong các `Blind_Cage` bao gồm các `Blind_Cell` chứa thông tin của từng ô.

Một số method đáng lưu ý của class này:

- Method khởi tạo __init__(self, n: int, cages: list [Blind_Cage])
- Method tìm ô bạn bè all_peers(self , row: int , col: int) –> set[Blind_Cell]. Method này nhận vào tọa độ của một cell gốc , và tìm tập hợp các cell, mà cell gốc "tiếp xúc" (tất nhiên là không kể cell gốc). "Tiếp xúc" được định nghĩa là cùng hàng, hoặc cùng cột, hoặc cùng khối (box), hoặc cùng vùng (cage) [1]

Bên ngoài class này, có hàm `solve_naive()` để giải bài toán. Khi chạy thuật toán `solve_naive()`, thuật toán sẽ đi tìm ô "đầu tiên" còn trống `_first_empty()`. Sau đó, thuật toán sẽ thử lần lượt các khả năng có thể từ 1 đến n.

[1] Một box là một tập hợp ô $n \times n$ trong một grid $n^2 \times n^2$, còn một cage là tập hợp các ô có tổng = target

Nếu như chọn được một giá trị thỏa mãn `_naive_ok()`, thuật toán đệ quy để chọn ô tiếp theo. Còn không, thuật toán sẽ thực hiện quay lui.

Cách chạy chương trình:

1. Input đầu vào:

```
PUZZLE_4x4 = [
    (7, [(0,0), (0,1), (0,2)]),
    (5, [(0,3), (1,3)]),
    (7, [(1,0), (1,1)]),
    (3, [(1,2), (2,2)]),
    (6, [(2,0), (3,0)]),
    (4, [(2,1), (3,1)]),
    (8, [(2,3), (3,2), (3,3)])
]
```

2. Lệnh chạy chương trình:

   - `python3 ten_file.py (Unix)`
   - `python ten_file.py (Windows)`.

3. Kết quả:

```
=======================================================
  KILLER SUDOKU — NAIVE SOLVER BENCHMARK
=======================================================

  4x4  ·  8 cages
  ---------------------------------------------------
  solver        result     time        peak memory
  ---------------------------------------------------
  naive         solved     0.0000s     2 KB

  Solution:

  1 2   4 3
  3 4   1 2

  4 3   2 1
  2 1   3 4


  9x9  ·  35 cages
  ---------------------------------------------------
  solver        result     time        peak memory
  ---------------------------------------------------
  naive         solved     21.9060s    6 KB

  Solution:

  2 4 3   9 6 5   8 7 1
  1 6 5   8 7 4   9 2 3
  7 8 9   2 1 3   4 6 5

  4 7 8   3 9 2   5 1 6
  6 9 2   4 5 1   3 8 7
  5 3 1   7 8 6   2 4 9

  3 5 7   6 4 8   1 9 2
  9 1 4   5 2 7   6 3 8
  8 2 6   1 3 9   7 5 4
```

Hình 4: Kết quả chạy chương trình Naive với 2 puzzle 4x4 và 9x9

2.3 Thuật toán tìm kiếm bằng kinh nghiệm

2.3.1 Ý tưởng

Giống như thuật toán trước, thuật toán này cũng sử dụng quay lui. Tuy nhiên thay vì thử một cách lần lượt qua tất cả các ô trong ma trận, thuật toán sẽ thử một cách "thông minh" hơn.
Vì có thể mỗi ô sẽ tồn tại những giá trị trong đoạn [1,N] nhưng không hợp lệ do những ràng buộc của trò chơi. Có nghĩa là số giá trị hợp lệ của mỗi ô sẽ khác nhau. Nên ta sẽ ưu tiên đi tìm những ô nào có số giá trị hợp lệ ít nhất, đồng nghĩa với ít lựa chọn nhất để điền đầu tiên. Bằng cách này, ta sẽ giới hạn số lần quay lui, giảm khả năng dẫn đến ngõ cụt và nhanh tìm giải pháp hơn.

2.3.2 Mô hình hóa bài toán

Mô hình hóa của bài toán trong giải thuật này được xây dựng tương tự của mô hình hóa thuật toán Naive.

Điều khác biệt lớn nhất là ở không gian tìm kiếm. Thay vì thử hết tất cả các cấu hình, nó lượt bỏ (prune) đi những cấu hình bất khả thi, giảm thiểu số lượng cấu hình xuống đáng kể.

2.3.3 Cải tiến so với thuật toán Naive

1. **Forward Checking và Pruning:**

   Khi một giá trị được điền vào ô, thì vùng (cage) chứa ô đó sẽ kiểm tra việc thêm cái giá trị còn lại có khả thi hay không. Ví dụ, có 3 ô với `target_sum = 24`. Nếu như điền 1 trong các ô là 2, thì 2 ô còn lại phải có tổng là 20 → Không thể với lưới 9x9 → Prune sớm.

2. **Minimum Remaining Value (MRV) heuristic**

   Thay vì chọn điền ô số 1 → điền ô số 2 → ... tuần tự đến khi tìm được lời giải, thì thuật toán sẽ chọn ô có ít giá trị nhất có thể, để giảm thiểu số nhánh khi quay lui.

2.3.4 Hiện thực

Môi trường hệ thống:

- **Ngôn ngữ**: `Python 3.14.3`
- **Thư viện**: `time, tracemalloc, functools`

Những thứ đáng nói trong mã nguồn:

1. Các class dữ liệu: `Heuristic_Cell` và `Heuristic_Cage`
2. Phương thức `is_valid_refined(cell, val)` kiểm tra tính hợp lệ khi điền một số nào đó vào `cell`. Nó sẽ trả về `False` nếu như tồn tại số trùng lặp trong hàng, cột, khối 3x3. Hoặc nếu như các số còn lại trong vùng (cage) là không thể điền được (Forward Checking)
3. Phương thức `refresh_valid_nums()` tính toán lại các dữ liệu mỗi khi có quay lui
4. Thuật toán tìm ô có ít giá trị khả thi nhất, được viết vào sẵn trong hàm `solve()`
5. Hàm `solve()`: Chọn ô có ít giá trị khả thi nhất, và thử điền lần lượt các giá trị có thể vào. Tiếp tục đệ quy đến khi giải được bài toán. Nếu như gặp một lối cụt, thì thực hiện quay lui.

Cách chạy chương trình:

- Input đầu vào: (bảng 9x9, xem thêm trong file)

```
    input_data = [
        (6, [(0,0), (0,1)]),
        (12, [(0,2), (0,3)]),
        (11, [(0,4), (0,5)]),
        (17, [(0,6), (0,7), (1,7)]),
        (4, [(0,8), (1,8)]),
        (7, [(1,0), (1,1)]),
        (16, [(1,2), (2,2), (2,3)]),
        (15, [(1,3), (1,4)]),
        ...
        (10, [(7,0), (7,1)]),
        (7, [(7,3), (7,4)]),
        (19, [(7,5), (8,4), (8,5)]),
        (27, [(7,7), (7,8), (8,6), (8,7), (8,8)]),
        (10, [(8,0), (8,1)]),
        (7, [(8,2), (8,3)])
    ]
```

- Lệnh chạy chương trình:
  - `python3 ten_file.py (Unix)`

– `python ten_file.py (Windows)`.

- Kết quả:

```
0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0
START SOLVING...
Đáp án:
2 4 3 9 6 5 8 7 1
1 6 5 8 7 4 9 2 3
7 8 9 2 1 3 4 6 5
4 7 8 3 9 2 5 1 6
6 9 2 4 5 1 3 8 7
5 3 1 7 8 6 2 4 9
3 5 7 6 4 8 1 9 2
9 1 4 5 2 7 6 3 8
8 2 6 1 3 9 7 5 4
-----------------------------
Time: 16.7500s
Peak Memory: 2456.40 KB
-----------------------------
```

Hình 5: Kết quả chạy chương trình Heuristic với puzzle 9x9

2.4 So sánh hiệu năng giữa 2 giải thuật

Với đầu vào là 4x4, ta có kết quả so sánh chạy trung bình như sau:

	Time	Peak memory
Naive	$\sim 0.5ms$	$\sim 2KB$
Heuristic	$\sim 0.38ms$	$\sim 65KB$

Với đầu vào là 9x9, ta có kết quả so sánh chạy trung bình như sau:

	Time	Peak memory
Naive	$\sim 19.38s$	$\sim 7KB$
Heuristic	$\sim 7.2918s$	$\sim 2400KB$

Nhận xét:

Thuật toán Naive chạy nhanh hơn Heuristic trong bảng 4x4, nhưng chậm hơn đáng kể nếu so bảng 9x9. Cả hai bảng đều cho thấy Heuristic dùng (rất) nhiều bộ nhớ hơn so với Naive.